

Magic Cards

Sistema interno de inventario de cartas coleccionables

Proyecto Final – Bases de Datos 1

Autores:

Angélica María Gómez Calderón (1000858150)

Andrés Camilo Pedraza Reyes (1098682948)

Daniel Felipe Gómez Veru (1021632658)

Docente:

Christian Felipe Duarte

UNIVERSIDAD EL BOSQUE

PROGRAMA DE INGENIERÍA DE SISTEMAS

FACULTAD DE INGENIERÍA

Bogotá, 2026

Tabla de contenido

1. RESUMEN / ABSTRACT 4
1.1 Resumen 4
1.2 Abstract 4
2. INTRODUCCIÓN 5
2.1 Contexto y problemática 5
2.2 Justificación 5
2.3 Objetivos 5
2.4 Alcance y limitaciones 6
2.5 Estructura del documento 6
3. MARCO TEÓRICO 7
3.1 Marco conceptual 7
3.1.1 Bases de datos relacionales y RDBMS 7
3.1.2 Modelo Entidad-Relación y modelo extendido 7
3.1.3 Normalización 7
3.1.4 Álgebra relacional y SQL 7
3.1.5 Virtualización, computación en la nube y acceso remoto 7
3.1.6 Control de versiones 7
3.3 Marco tecnológico 8
3.4 Marco legal 8
4. MODELO BIOPSICOSOCIAL Y CULTURAL 9
4.1 Situación identificada a intervenir 9
4.2 Análisis desde el modelo biopsicosocial y cultural 9
4.3 Componente de análisis: transformación esperada del contexto 9
5. DISEÑO, IMPLEMENTACIÓN Y RESULTADOS 11
5.1 Diseño y modelado de datos (Fase 1) 11
5.2 Infraestructura: máquina virtual y RDBMS (Fase 2) 11
5.3 Scripts, aplicación web y repositorio 12
5.4 Discusión de resultados 12
6. GESTIÓN DEL PROYECTO 13
6.1 Acta de constitución e interesados 13
6.2 EDT/WBS y cronograma 13
6.3 Presupuesto 13
6.4 Riesgos 13
6.5 Cierre y compromisos posteriores 14
7. CONCLUSIONES Y LECCIONES APRENDIDAS 15
7.1 Conclusiones 15
7.2 Lecciones aprendidas 15
8. ANEXOS 16
Anexo A: Análisis de selección de sistema operativo 17

Anexo B: Implementación de la máquina virtual (EC2).....20
Anexo C: Análisis de selección del RDBMS.....28
Anexo D: Implementación del RDBMS.....31
Anexo E: Acta de constitución del proyecto (incluye interesados, cronograma y presupuesto).34
Anexo F: Acta de cierre.....40
9. REFERENCIAS 45

1. RESUMEN / ABSTRACT

1.1 Resumen

Magic Cards es un sistema de información interno para una tienda especializada en cartas coleccionables de Magic desarrollado como proyecto final de la asignatura Bases de Datos 1. El problema que tiene es la dispersión de la información en este tipo de negocios, que genera problemas en el inventario como reservar el mismo ejemplar a dos clientes, perder el rastro de a quién se entregó cada carta o desconocer cuánto vale lo que hay en inventario. Para resolverlo se diseñó un Modelo Entidad-Relación Extendido de 17 entidades (15 regulares y 2 débiles) y 20 relaciones, que se tradujo a un modelo relacional normalizado hasta la Tercera Forma Normal (3FN) y a un esquema físico de 20 tablas en MySQL.

La base de datos se desplegó en una instancia de Amazon EC2 (AWS) y se accede a ella desde la aplicación web. Las consultas del sistema se formularon en álgebra relacional (mínimo 50 sentencias) y el código y la documentación se manejaron con GitHub. Como resultado se completaron los entregables de las dos fases del proyecto y los entregables transversales de gestión, entre ellos el acta de constitución y el acta de cierre.

Palabras clave: bases de datos relacionales, modelo entidad-relación, normalización, álgebra relacional, MySQL, Amazon EC2, inventario, cartas coleccionables.

1.2 Abstract

Magic Cards is an internal information system for a shop specializing in *Magic* trading cards, developed as a final project for the "Databases 1" course. A common challenge in this type of business is the fragmentation of information, which leads to inventory issues—such as reserving the same item for two different customers, losing track of which customer received a specific card, or being unable to determine the total value of the current inventory. To address this, an Extended Entity-Relationship model was designed, comprising 17 entities (15 regular and 2 weak) and 20 relationships. This was subsequently translated into a relational model—normalized to the Third Normal Form (3FN)—and implemented as a physical schema consisting of 20 tables in MySQL.
The database was deployed on an Amazon EC2 (AWS) instance and is accessed via a web application. System queries were formulated using relational algebra (with a minimum of 50 statements), and both code and documentation were managed using GitHub. The project deliverables for both phases were completed, along with cross-functional management documentation, including the project charter and the project closure report.
Keywords: relational databases, entity-relationship model, normalization, relational algebra, MySQL, Amazon EC2, inventory, trading cards.

2. INTRODUCCIÓN

2.1 Contexto y problemática

Una tienda especializada en cartas coleccionables maneja un inventario particular: cada ejemplar es distinto de los demás por su condición física, su edición y su idioma y su valor cambia con el tiempo según el precio de referencia. Cuando la información de un negocio así está dispersa en libretas y hojas de cálculo aparecen riesgos como reservar el mismo ejemplar a dos clientes, perder el rastro de a quién se le entregó cada carta o no saber cuánto vale lo que hay en inventario.

Magic_Cards modela el inventario de una tienda de Magic: ejemplar por ejemplar: qué cartas tiene, en qué condición está cada copia, cuánto vale según su precio de referencia y si está disponible o apartada. Los clientes registrados pueden reservar ejemplares desde su perfil, con un plazo máximo de 30 días para reclamarlos; vencido el plazo, la reserva se libera automáticamente. Cada entrega de ejemplares a un cliente queda registrada como una salida del inventario, con el cliente identificado y el empleado responsable.

2.2 Justificación

El problema es lo bastante difícil para aplicar los conceptos de la asignatura (relaciones uno a uno, uno a muchos y muchos a muchos, entidades débiles, una relación recursiva, atributos derivados y compuestos, y normalización a 3FN) Además, resuelve una necesidad real de control trazabilidad de cada ejemplar, reglas claras de reserva y una forma confiable de conocer el valor del inventario.

2.3 Objetivos

Objetivo general. Diseñar, implementar y documentar una base de datos relacional normalizada hasta la Tercera Forma Normal, desplegada en una máquina virtual Linux en AWS y consultable desde una aplicación web, que gestione el inventario y las reservas de la tienda.

Objetivos específicos.

1. Modelar entidad-relación en notación Peter Chen y Crow's Foot, con al menos 8 entidades (el modelo desarrollado tiene 17), usando DRAIWO.
2. Realizar el modelo relacional cumpliendo con las normas de transformación y realizando normalización hasta que estén en 3FN y describirlo en un diccionario de datos, Usando Oracle SQL Developer Data Modeler
3. Escribir al menos 50 sentencias de álgebra relacional que demuestren las consultas del sistema.
4. Seleccionar y justificar el sistema operativo y el RDBMS, e implementarlos en una máquina virtual.
5. Pasar del modelo lógico al físico con scripts de creación de objetos, inserción de registros y scripts varios
6. Construir un sistema de información sencillo conectada a la base de datos, con lectura de datos.
7. Trabajar en GitHub con aportes significativos de los tres integrantes.

8. Entregar el documento final, la presentación y un video de demostración de máximo 15 minutos con la participación de todos.

2.4 Alcance y limitaciones

El proyecto incluye:

- Catálogo de cartas con colores, tipos, ediciones, rarezas, artistas, idiomas y linaje de reimpresiones.
- Inventario por ejemplar, con condición física, fecha de ingreso y estado.
- Historial de precios de referencia y cálculo del precio sugerido.
- Tres perfiles de usuario: cliente, empleado y administrador.
- Módulo de reservas con plazo de 30 días y liberación automática.
- Registro de salidas, con el cliente identificado y el empleado responsable.
- Consultas y agregaciones: valor del inventario, disponibilidad real y vencimiento de reservas.
- Un sistema de información de demostración.

El proyecto no incluye: ventas, compras, consignaciones, intercambios, pagos y facturación; varias sucursales, ventas en línea y cotizaciones en tiempo real; formatos de juego, mazos, reglas de interacción y erratas; proveedores y procedencia de la mercancía; traslados, inventarios físicos periódicos, devoluciones, garantías y domicilios; listas de espera, notificacionesLos datos de cartas, precios y usuarios son de prueba, y los precios de referencia se cargan a mano.

2.5 Estructura del documento

La sección 3 presenta el marco teórico; la sección 4, el análisis desde el Modelo Biopsicosocial y Cultural; la sección 5, el diseño, la implementación y los resultados; la sección 6, la gestión del proyecto; y la sección 7, las conclusiones y lecciones aprendidas. Los anexos completos se incluyen a continuación del documento principal, relacionados en la sección 9.

3. MARCO TEÓRICO

3.1 Marco conceptual

3.1.1 Bases de datos relacionales y RDBMS

El modelo relacional, propuesto por Codd, representa los datos como relaciones (tablas) formadas por filas y columnas, y se apoya en el álgebra relacional para manipularlos. Un sistema gestor de bases de datos relacional (RDBMS) implementa este modelo y garantiza integridad, consistencia y acceso concurrente a los datos. Los tipos de integridad que sustentan el diseño son: integridad de entidad (claves primarias).

3.1.2 Modelo Entidad-Relación y modelo extendido

El modelo Entidad-Relación, introducido por Chen, describe el dominio mediante entidades, atributos y relaciones con su cardinalidad (uno a uno, uno a muchos y muchos a muchos). La versión extendida incorpora conceptos como entidades débiles, que dependen de otra para identificarse; relaciones recursivas, que enlazan una entidad consigo misma; y atributos compuestos y derivados. En este proyecto el modelo se dibujó en notación Peter Chen.

3.1.3 Normalización

La normalización organiza las tablas para eliminar redundancia y evitar anomalías de inserción, actualización y eliminación. Un esquema está en Tercera Forma Normal (3FN) cuando todos sus atributos dependen únicamente de la clave y no existen dependencias transitivas.

3.1.4 Álgebra relacional y SQL

El álgebra relacional define operadores sobre relaciones: selección, proyección, unión, intersección, diferencia, producto cartesiano, reunión (join) y renombramiento. Cada uno tiene un equivalente directo en SQL: la selección corresponde a WHERE, la proyección a SELECT, la reunión a JOIN, la unión a UNION, la intersección a INTERSECT, el producto a CROSS JOIN y el renombramiento a un alias.

3.1.5 Virtualización, computación en la nube y acceso remoto

Una máquina virtual (VM) es un computador simulado por software sobre hardware físico, gestionado por un hipervisor. Amazon EC2 ofrece máquinas virtuales como infraestructura como servicio: el proveedor aporta el hardware y el hipervisor, y el usuario instala y administra el sistema operativo y las aplicaciones. Esto difiere de un servicio administrado como Amazon RDS, donde el proveedor instala y mantiene también el motor de base de datos; por eso, para poder documentar de verdad la implementación de la VM y del RDBMS, el proyecto usa EC2. El sistema operativo de la instancia es Amazon Linux 2023. El acceso se realiza por SSH con un par de llaves, usando el cliente OpenSSH incluido en Windows.

3.1.6 Control de versiones

Git es un sistema de control de versiones distribuido que registra el historial de cambios del código y la documentación, y GitHub es la plataforma que aloja el repositorio remoto. En el proyecto permite evidenciar los aportes de cada integrante y ejecutar los mismos scripts en cualquier máquina del equipo.

3.3 Marco tecnológico

Componente	Herramienta	Uso en el proyecto
Modelado	,Drawio y Oracle SQL Developer Data Modeler	Modelos entidad-relación y relacional en notación Crow's Foot
RDBMS	MySQL Community Server	Base de datos magic_cards, con claves primarias compuestas, claves foráneas
Cliente de base de datos	MySQL Workbench	Administración y consulta de la base desde Windows
Máquina virtual	Amazon EC2, Amazon Linux 2023	Servidor administrado por el equipo, instancia t3.micro
Acceso remoto	OpenSSH (PowerShell)	Conexión a la instancia y túnel SSH hacia la base de datos
Control de versiones	GitHub	Repositorio con scripts, documentación y aplicación web

3.4 Marco legal

Los datos de cartas, precios y usuarios de este proyecto son datos de prueba, por lo que no se almacenan datos personales reales; aun así, el diseño contempla perfiles de acceso diferenciados y contraseñas, de acuerdo con el principio de mínimo privilegio.

4. MODELO BIOPSICOSOCIAL Y CULTURAL

El modelo biopsicosocial, sostiene que un problema humano no tiene una sola causa, sino que lo influyen de manera simultánea factores biológicos, psicológicos y sociales; el enfoque se complementa aquí con la dimensión cultural. Aplicarlo a un proyecto de ingeniería permite analizar no solo qué resuelve la tecnología, sino cómo cambia el contexto de las personas que la usan.

4.1 Situación identificada a intervenir

Actores. El sistema involucra tres perfiles: el **cliente**, que consulta el catálogo, se registra y crea y cancela sus propias reservas; el **empleado**, que registra el ingreso de ejemplares, actualiza su condición y ubicación, carga los precios de referencia, registra las salidas y marca las reservas como reclamadas o vencidas; y el **administrador**, que hace todo lo del empleado y además administra los catálogos, los usuarios, los roles y los empleados.

Situación actual. La información de una tienda de este tipo suele estar dispersa, lo que trae tres riesgos: reservar el mismo ejemplar a dos clientes, perder el rastro de a quién se le entregó cada carta y no saber cuánto vale el inventario. Estas fallas afectan la confianza del cliente y generan trabajo manual y reprocesos para el personal.

Situación propuesta. Con Magic_Cards, cada ejemplar tiene un estado controlado (disponible, reservado, entregado o dado de baja), las reservas tienen un plazo máximo de 30 días tras el cual se liberan automáticamente, cada salida queda asociada a un cliente y a un empleado responsable, y el valor del inventario se calcula a partir de los precios de referencia.

4.2 Análisis desde el modelo biopsicosocial y cultural

La siguiente tabla analiza la situación actual en cada dimensión. Se trata de un análisis construido a partir del problema definido en el proyecto, no de una medición con usuarios reales.

Dimensión	Análisis de la situación actual
Biológica	Dimensión de alcance limitado en este proyecto, pues el problema no implica riesgos directos para la salud. Se considera el desgaste del personal por llevar controles manuales y el cuidado físico de los ejemplares, cuya condición determina su valor.
Psicológica	Desconfianza y frustración del cliente cuando una carta que creía apartada ya no está disponible; incertidumbre del personal sobre el estado real del inventario; y el valor afectivo que el coleccionista asigna a cada carta.
Social	Relación cliente-tienda sin trazabilidad: no queda registro claro de quién reservó, quién recibió y qué empleado entregó. La comunidad de jugadores y coleccionistas depende de la confianza en el trato de la tienda.
Cultural	El coleccionismo es una práctica con valores propios (rareza, condición, ediciones). Es común apartar cartas de palabra o llevar el control en medios informales, costumbres que dificultan la estandarización.

4.3 Componente de análisis: transformación esperada del contexto

Dimensión	Transformación esperada con Magic_Cards
Biológica	Menor carga de trabajo manual y mejor cuidado de los ejemplares al registrar su condición y ubicación exactas.

Dimensión	Transformación esperada con Magic_Cards
Psicológica	Mayor confianza y sensación de control: el cliente sabe hasta cuándo tiene su reserva y el personal conoce el estado real de cada ejemplar.
Social	Relación más transparente entre cliente y tienda: cada reserva y cada salida queda registrada con su cliente y su empleado responsable.
Cultural	Paso de prácticas informales a un registro estandarizado, que respeta el valor del coleccionismo y fomenta la formalidad en el trato.

En términos prácticos, se espera eliminar la doble reserva de un mismo ejemplar, conservar el historial de cada entrega y contar con una medida confiable del valor del inventario.

5. DISEÑO, IMPLEMENTACIÓN Y RESULTADOS

Esta sección resume el trabajo realizado en las dos fases del proyecto. El detalle completo de cada entregable se encuentra en el anexo indicado.

5.1 Diseño y modelado de datos (Fase 1)

El modelo conceptual (MERE) representa fielmente la operación interna de la tienda según los supuestos definidos. Incluye una relación recursiva con atributo propio (REIMPRIMIR, que enlaza una impresión consigo misma), entidades débiles (IMPRESION, identificada por edición, idioma, número de colección e indicador foil; y PRECIO_REFERENCIA, identificada por la impresión y la fecha), atributos derivados que no se almacenan (precio sugerido, fecha de vencimiento de la reserva, valor de maná y disponibilidad real) y atributos compuestos. El modelo se normalizó hasta la 3FN y se describió en un diccionario de datos. Los modelos, el diccionario de datos y las sentencias de álgebra relacional son entregables de la Fase 1 y se encuentran en el repositorio del proyecto

Indicador	Resultado
Entidades del MERE	17
Relaciones	20
Normalización	Tercera Forma Normal (3FN)
Tablas del esquema físico	20, con claves primarias compuestas y claves foráneas
Sentencias de álgebra relacional	Mínimo 50 (repositorio)

Entre las reglas de negocio del modelo están las relaciones uno a uno entre cuentas, clientes y empleados, y entre reservas y salidas; los estados controlados de ejemplares y reservas; y la regla de que un ejemplar solo puede estar en una reserva vigente a la vez.

5.2 Infraestructura: máquina virtual y RDBMS (Fase 2)

Evolución de las decisiones técnicas. En los documentos de análisis de la Fase 2 (Anexos A y C) el equipo comparó cuatro distribuciones Linux y cuatro motores relacionales (Oracle Database Free, PostgreSQL, MySQL Community y MariaDB), y propuso como configuración de referencia AWS Linux, por su facilidad de instalación ya que es simplemente inscribirse sin ninguna descarga anteriormente y porque todo queda en la nube esto siendo mas practico para que los compañeros se conecten a una misma bd. En la implementación (Anexos B y D), la instancia se lanzó con Amazon Linux 2023 y el motor desplegado fue MySQL Community Server.

Ambas configuraciones cumplen las restricciones del enunciado (distribución Linux, RDBMS distinto de SQL Server y ejecución en una máquina virtual), y el modelo relacional en 3FN es independiente del motor elegido.

Máquina virtual (Anexo B). Se creó una cuenta de AWS con el plan gratuito y se lanzó una instancia Amazon EC2 de tipo t3.micro (2 vCPU y 1 GiB de memoria) con la imagen Amazon Linux 2023, un volumen EBS gp3, asignación de IP pública y un par de claves RSA en formato .pem. El grupo de seguridad restringe el acceso SSH a la dirección IP del equipo. La administración se hace por SSH desde PowerShell, con el usuario ec2-user.

RDBMS (Anexo D). Sobre la instancia se agregó el repositorio oficial de MySQL, se instaló MySQL Community Server con el gestor de paquetes dnf y se iniciaron y habilitaron el servicio mysqld con systemctl, para que arranque automáticamente al reiniciar la máquina. Se cargó la base de datos magic_cards y se creó un usuario de aplicación con permisos limitados. El acceso desde Windows se hace por un túnel SSH, de modo que el puerto de la base de datos no queda expuesto a internet, y la base se administra con MySQL Workbench.

5.3 Scripts, aplicación web y repositorio

La transición del modelo lógico al físico se documentó con tres bloques de scripts y su README: creación de objetos, inserción de datos de prueba y creación del usuario de la aplicación. Los scripts se probaron en orden sobre un servidor limpio, lo que permitió detectar errores antes de la entrega (scripts y README en el repositorio). La aplicación web de demostración se conecta a la base de datos y lee datos reales de ella. Todo el código y la documentación están en el repositorio de GitHub del proyecto, que evidencia los aportes de los tres integrantes, y el proceso se muestra en un video de demostración de máximo 15 minutos con la participación de todos.

5.4 Discusión de resultados

La siguiente tabla relaciona cada objetivo específico con su evidencia. Según el acta de cierre, los entregables de las dos fases y los transversales se completaron sin pendientes.

Objetivo específico	Evidencia	Estado
1. Modelos E-R (Chen y Crow's Foot)	\Desktop\Proyecto_Final_BD1\FASE 01	Completado
2. Modelo relacional en 3FN y diccionario	\Desktop\Proyecto_Final_BD1\FASE 01	Completado
3. Mínimo 50 sentencias de álgebra relacional	\Desktop\Proyecto_Final_BD1\FASE 01	Completado
4. Selección e implementación de SO, VM y RDBMS	Anexos A, B, C y D	Completado
5. Scripts con README	\Desktop\Proyecto_Final_BD1\FASE 02	Completado
6. Aplicación web conectada a la base de datos	Repositorio	Completado
7. Trabajo en GitHub	\Desktop\Proyecto_Final_BD1\FASE 01 (LINK)	Completado
8. Documento final, presentación y video	\Desktop\Proyecto_Final_BD1\FASE 01	Completado

6. GESTIÓN DEL PROYECTO

6.1 Acta de constitución e interesados

El acta de constitución (Anexo E) formaliza el proyecto: el proyecto final de Bases de Datos 1 representa el 20 % de la nota del semestre y se desarrolló durante 14 semanas del periodo 2026-2, entre el 4 de agosto y el 2 de noviembre de 2026. El interesado principal es el docente Christian Felipe Duarte, quien actúa como patrocinador y evaluador y valora el dominio de los conceptos del curso. Le sigue el equipo, integrado por Angélica María Gómez Calderón, Andrés Camilo Pedraza Reyes y Daniel Felipe Gómez Veru, que participó en todas las fases. Por último están los usuarios del sistema, que en este caso son ficticios pero guían el diseño: el cliente que consulta y reserva, y el personal de la tienda (empleados y administrador), que necesita información confiable. El análisis de interesados se resume en el acta de constitución (Anexo E).

6.2 EDT/WBS y cronograma

La EDT/WBS y el cronograma detallado se entregan como documentos transversales; el acta de constitución (Anexo E) los resume. Como referencia, las 14 semanas se repartieron así:

Semanas	Actividades
1 y 2	Inicio, propuesta, supuestos y acta de constitución
3 a 6	Modelos entidad-relación y relacional, y diccionario de datos
6 a 8	Sentencias de álgebra relacional
8 a 10	Instancia en AWS, instalación del motor y documentos de análisis e implementación
10 a 12	Scripts de creación, inserción y varios, con su README
12 a 14	Aplicación web conectada, video, documento final, presentación y acta de cierre

6.3 Presupuesto

No se pagaron licencias de software: Drawio,Oracle SQL Developer Data Modeler, MySQL Community Server, Git y GitHub son gratuitos para este uso. El costo real está en AWS (instancia EC2, disco y transferencia de datos), que se cubre con los créditos de la cuenta nueva de AWS; en la captura de la consola se observan USD 100 en créditos. Para controlar el gasto, la instancia solo se mantiene encendida cuando se necesita. El detalle valorado se encuentra en el entregable de presupuesto y se resume en el acta de constitución (Anexo E).

6.4 Riesgos

Riesgo	Respuesta prevista
Modelo demasiado ambicioso para el tiempo disponible	Mantener el alcance acotado por los supuestos y priorizar la lectura de datos en la web
Aportes desiguales entre los integrantes	Reuniones semanales, tareas asignadas en GitHub y revisión de los commits
Consumo de AWS mayor al previsto	Apagar la instancia cuando no se usa y revisar el panel de costos cada semana
Fallas en la instalación del motor o en la conexión de la web	Documentar cada paso y probar la conexión desde el inicio de la Fase 2

Riesgo	Respuesta prevista
Regla de un ejemplar con una sola reserva vigente mal implementada	Controlarla
Carga académica que retrase las entregas	Cronograma con holguras

6.5 Cierre y compromisos posteriores

El acta de cierre (Anexo F) registra el estado de los entregables, los resultados, las lecciones aprendidas y las firmas del equipo. Como compromisos posteriores al cierre, el equipo mantendrá disponible el repositorio con los scripts, la documentación y la aplicación; conservará una copia de la base de datos y de los scripts fuera de AWS; y, al terminar la sustentación, detendrá o eliminará la instancia EC2 y los recursos asociados para evitar cargos o consumo de créditos.

7. CONCLUSIONES Y LECCIONES APRENDIDAS

7.1 Conclusiones

El proyecto permitió diseñar, implementar y documentar una base de datos relacional para una tienda de cartas coleccionables, desde el modelo conceptual hasta el despliegue en una máquina virtual Linux en la nube y su consulta desde una aplicación web. Se completaron los entregables de las dos fases y los transversales de gestión.

En el diseño, el modelo de 17 entidades y 20 relaciones, normalizado hasta la 3FN, permitió representar las reglas de negocio de la tienda: estados controlados, reservas con plazo y liberación automática, y salidas con cliente y empleado responsable. En la implementación, el esquema físico de 20 tablas y los scripts ejecutables en orden sobre un servidor limpio demostraron que el modelo lógico se traduce de forma reproducible a un motor real. En la infraestructura, el acceso por túnel SSH, sin exponer el puerto de la base de datos, aplicó el principio de mínimo privilegio también a nivel de red.

Finalmente, el análisis biopsicosocial y cultural indica que el valor del sistema no es solo técnico: al registrar cada reserva y cada entrega se espera aumentar la confianza entre cliente y tienda.

7.2 Lecciones aprendidas

Lo que funcionó bien:

- La división del trabajo por fases permitió avanzar de forma organizada entre el modelado conceptual y la implementación técnica.
- El documento de supuestos, con sus reglas de negocio y su resumen estructural, sirvió como fuente de verdad para el modelo y los scripts.
- El repositorio Git como herramienta central garantizó el control de versiones.
- Probar los scripts en orden sobre un servidor limpio permitió detectar errores antes de la entrega.

Áreas de mejora:

- Mayor planificación inicial del tiempo para cada entregable, especialmente en la fase de implementación.
- Documentación progresiva durante el desarrollo, en lugar de concentrarla en las etapas finales.
- Revisar desde el inicio las condiciones de costo y vencimiento de la plataforma en la nube antes de elegirla.

8. ANEXOS

A continuación de este documento principal se incluyen los anexos completos, tal como fueron elaborados por el equipo. Cada anexo se referencia en las secciones anteriores. Los entregables de la Fase 1 (modelos, diccionario de datos y álgebra relacional), los scripts, la aplicación web y el video se encuentran en el repositorio del proyecto.

Anexo	Documento
A	Análisis de selección de sistema operativo
B	Implementación de la máquina virtual (EC2)
C	Análisis de selección del RDBMS
D	Implementación del RDBMS
E	Acta de constitución del proyecto (incluye interesados, cronograma y presupuesto)
F	Acta de cierre

Anexo A: Análisis de selección de sistema operativo

1. El problema que vamos a resolver

El sistema modela el inventario de una tienda de cartas coleccionables de Magic: The Gathering ejemplar por ejemplar: qué cartas tiene, en qué condición está cada copia, cuánto vale según su precio de referencia y si está disponible o apartada. Los clientes registrados pueden reservar ejemplares desde su perfil en la página, con un plazo máximo de 30 días para reclamarlos; vencido el plazo, la reserva se libera automáticamente. Cada entrega de ejemplares a un cliente queda registrada como salida del inventario.

El sistema no maneja ventas, compras, pagos ni facturación. Tiene tres perfiles de usuario: cliente, empleado y administrador. El enunciado completo y los supuestos del modelo están en el archivo Propuesta_Proyecto_BD1.pdf de nuestro repositorio de GitHub.

2. Qué necesita el servidor

Necesitamos un servidor que esté disponible cuando lo necesitemos, para que la aplicación web consulte la base de datos y para poder hacer la demostración. Necesitamos que aguante un motor relacional con 17 entidades y 20 relaciones, con claves compuestas y reglas de negocio que no se pueden romper. Necesitamos que sea seguro, porque hay tres perfiles de acceso y contraseñas. Y necesitamos poder instalarlo y volver a levantarlo siguiendo una guía escrita, porque los tres vamos a tocarlo y todos los scripts viven en GitHub.

No necesitamos rendimiento extremo: es una sola tienda con un volumen de datos moderado. Lo que sí necesitamos es estabilidad y que el sistema operativo no nos moleste

3. Las opciones que miramos

Comparamos cuatro distribuciones que se usan de verdad en servidores. Descartamos de entrada los Unix propietarios (Solaris, AIX, HP-UX) y los BSD, porque exigen hardware o conocimientos que no están en el alcance del curso y tienen mucha menos documentación para lo que vamos a hacer.

Oracle Linux 9. Es compatible con Red Hat Enterprise Linux, se descarga y se usa sin costo, y la mantiene Oracle, la misma empresa que fabrica la base de datos que pensamos instalar. Eso tiene una consecuencia práctica importante: es el sistema sobre el que Oracle prueba su motor, y trae un paquete de preinstalación que crea el usuario, ajusta los límites y configura los parámetros del kernel que la base de datos necesita. Esos pasos, en otra distribución, habría que hacerlos a mano y con más riesgo de equivocarnos. Su punto débil es que tiene menos tutoriales que Ubuntu y que requiere acostumbrarse a dnf, firewalld y SELinux.

Ubuntu Server 24.04 LTS. Es la más fácil de instalar y la que más ayuda tiene en internet. Si solo midiéramos comodidad, ganaría. Pero no es una plataforma soportada de forma oficial para el motor que elegimos, y se aparta del entorno que propone el curso.

Rocky Linux 9. Pertenece a la misma familia que Oracle Linux. Es sólida, pero no nos aporta nada que Oracle Linux no tenga, y no coincide con la referencia del enunciado.

Debian 12. Es muy estable y liviana, pero valen para ella las mismas observaciones que para Ubuntu: es buena distribución, aunque no es la plataforma natural para una base de datos Oracle.

4. **Dónde corre: una instancia EC2 en AWS**

Vamos a trabajar en AWS. Nuestra máquina virtual es una instancia EC2, que técnicamente es una VM (infraestructura como servicio): AWS pone el hardware y el hipervisor, y nosotros instalamos y administramos todo lo que va encima, es decir, el sistema operativo, los usuarios, el firewall, los paquetes y la base de datos.

Dejamos explícito que no usaremos servicios administrados como Amazon RDS. En un servicio administrado, AWS instala y mantiene el sistema operativo y el motor, y entonces no tendríamos nada real que documentar en los entregables de implementación de la VM y del RDBMS, que es justamente lo que el enunciado nos pide demostrar.

Para el tamaño de la máquina proponemos 2 vCPU y 1 GB de memoria (una instancia tipo t3.micro) con un disco de unos 20 GB. La edición gratuita usa como máximo 2 GB de memoria para la base de datos, y el sistema operativo y la aplicación web también necesitan su parte; una instancia de 2 GB funcionaría pero iría muy justa. Accederemos por SSH con un par de llaves, con el usuario ec2-user que traen estas imágenes.

Hay tres cuidados prácticos que queremos dejar anotados:

- **Seguridad de red.** El grupo de seguridad solo permitirá SSH desde nuestras direcciones IP y tráfico web (80 y 443) para la aplicación. El puerto de la base de datos (1521) no se abrirá a internet; la aplicación se conecta a ella desde dentro de la misma máquina.
- **Costos.** Las cuentas nuevas de AWS desde julio de 2025 reciben créditos en lugar de horas gratuitas, y algunas imágenes de Oracle Linux del Marketplace cobran un valor por hora por soporte. Antes de lanzar la instancia hay que revisar el costo de la imagen y, cuando no estemos trabajando, apagar la instancia. Ese gasto entra en el presupuesto del proyecto.
- **Respaldo.** Antes de cambios grandes tomaremos una captura (snapshot) del disco, y todo lo que se pueda reconstruir, como los scripts, estará en GitHub.

5. **Cómo se conecta esto con lo que aprendimos**

No elegimos el sistema operativo en el vacío; la decisión sale de cosas que vimos en la materia. El modelo entidad-relación y la normalización a 3FN nos dejaron muchas tablas con claves foráneas, algunas compuestas, y eso exige un servidor estable que haga cumplir la integridad referencial sin fallos. tener un entorno propio y no un servicio administrado nos deja ver cómo pasa la teoría a la práctica. El control de versiones con GitHub cobra sentido cuando los scripts se ejecutan igual en cualquier máquina del equipo.

6. **Riesgos que vemos**

• Que la instancia se quede encendida y consuma los créditos de AWS. Lo manejamos con una rutina de apagado y revisando el panel de costos cada semana.

• Que cada integrante configure su acceso de forma distinta. Lo manejamos con una guía única en el repositorio y llaves SSH por persona.

• Que la imagen descargada no sea la correcta. Lo manejamos verificando la versión y la fuente antes de usarla.

7. Conclusión

Oracle Linux 9, ejecutándose en una instancia EC2 que administramos nosotros, es la opción que mejor se ajusta a este proyecto: cumple el enunciado, es la plataforma natural de la base de datos que vamos a usar, es estable y segura, y nos permite documentar de verdad la implementación de la VM. Con esta decisión tomada, el siguiente paso es seleccionar e instalar el RDBMS, que explicamos en el documento siguiente.

Anexo B: Implementación de la máquina virtual (EC2)

1. Introducción

Este documento describe cómo crear y configurar la máquina virtual Linux en la nube de Amazon Web Services (AWS). La máquina virtual es una instancia de Amazon EC2 con Ubuntu Server 24.04 LTS, administrada por SSH desde Windows.

Versiones y recursos usados en esta guía:

- Ubuntu Server 24.04 LTS
- Instancia EC2 tipo t3.micro (2 vCPU, 1 GB de RAM)
- Disco EBS de 20 GB
- Cliente SSH incluido en Windows 10 y 11 (OpenSSH, desde PowerShell)

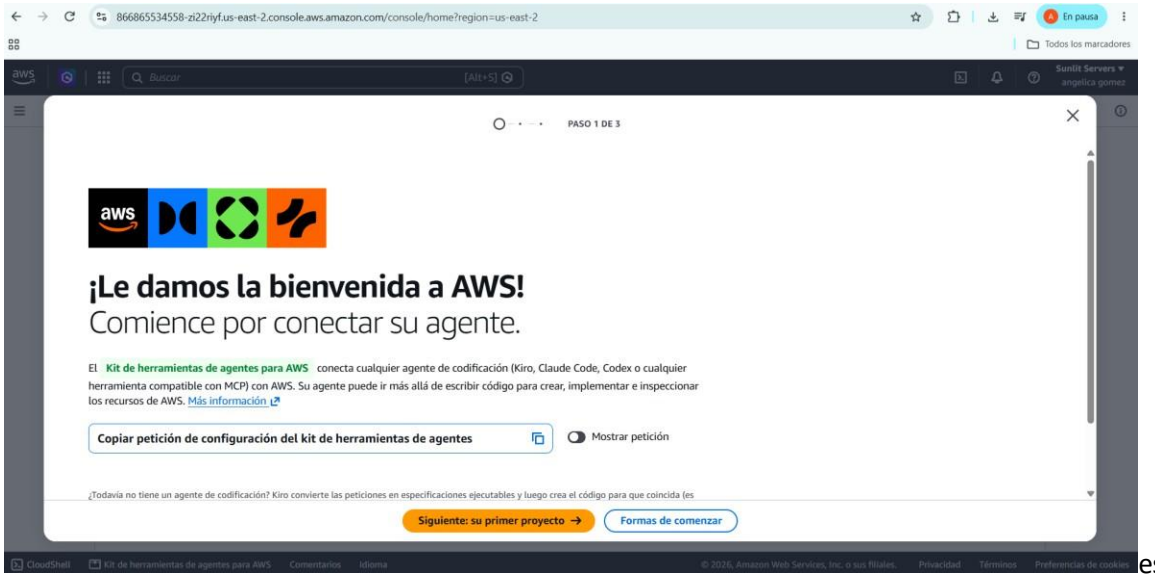
2. Qué necesita antes de empezar

- Un correo electrónico para la cuenta de AWS.
- Una tarjeta de débito o crédito, que AWS pide para verificar la cuenta.
- Windows con PowerShell.

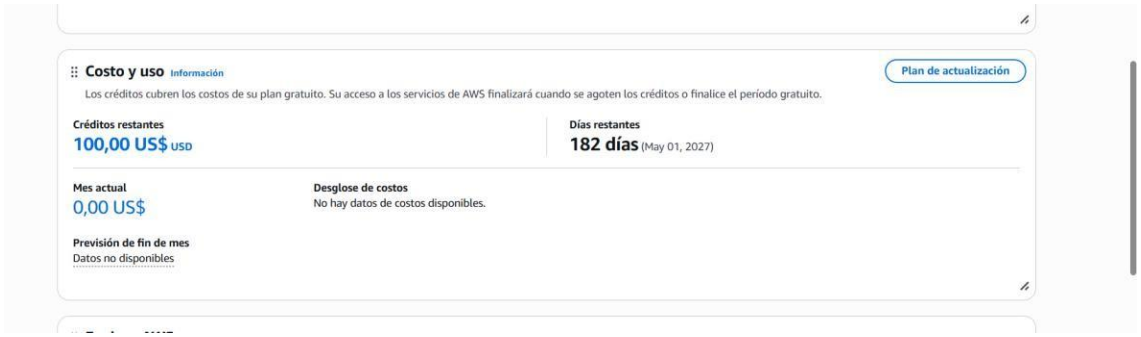
3. Crear la cuenta de AWS

- Entre a https://aws.amazon.com/free/ y pulse **Create a Free Account**.
- Complete el registro con el correo, el nombre de la cuenta y los datos de contacto.
- Cuando AWS ofrezca elegir un plan, seleccione **Free plan** (plan gratuito). A las cuentas nuevas les entrega créditos de uso y, mientras esté en el plan gratuito, no cobra a la tarjeta si se agotan.
- Inicie sesión en la consola de AWS.



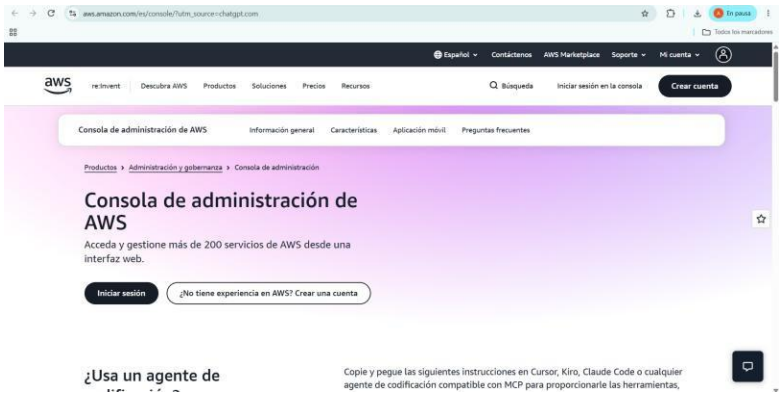


espa



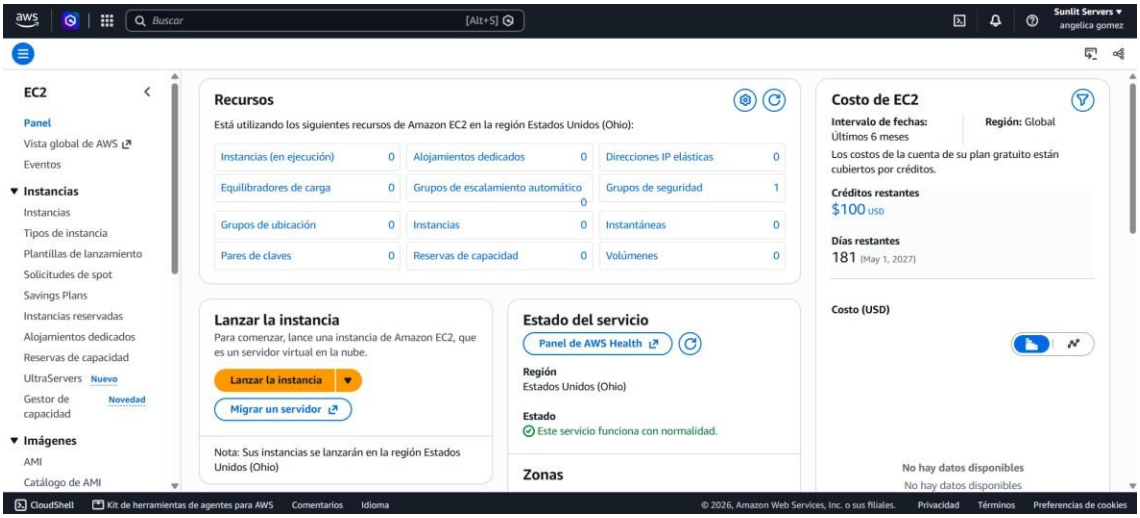
4.Ingresar a AWS

- Ingresa a AWS Management Console.
- Inicia sesión con tu cuenta.
- En la barra de búsqueda escribe EC2.
- Selecciona el servicio EC2 (Elastic Compute Cloud).

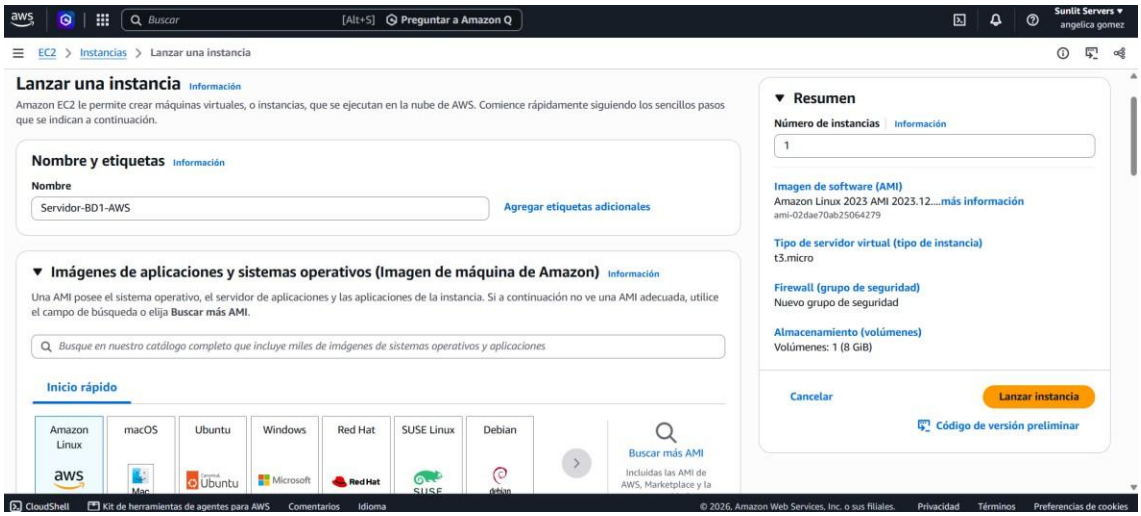


5. Crear una Instancia

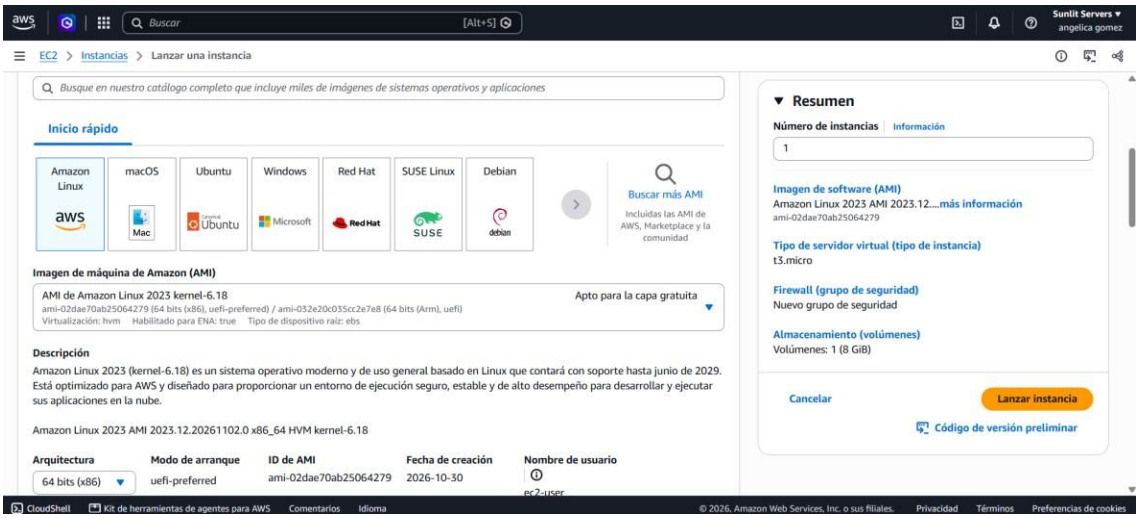
- En el menú izquierdo busca Instancias (Instances).
- Haz clic en Lanzar instancias (Launch instances).
- Se abrirá un formulario para configurar el servidor.



5. Asignar un nombre



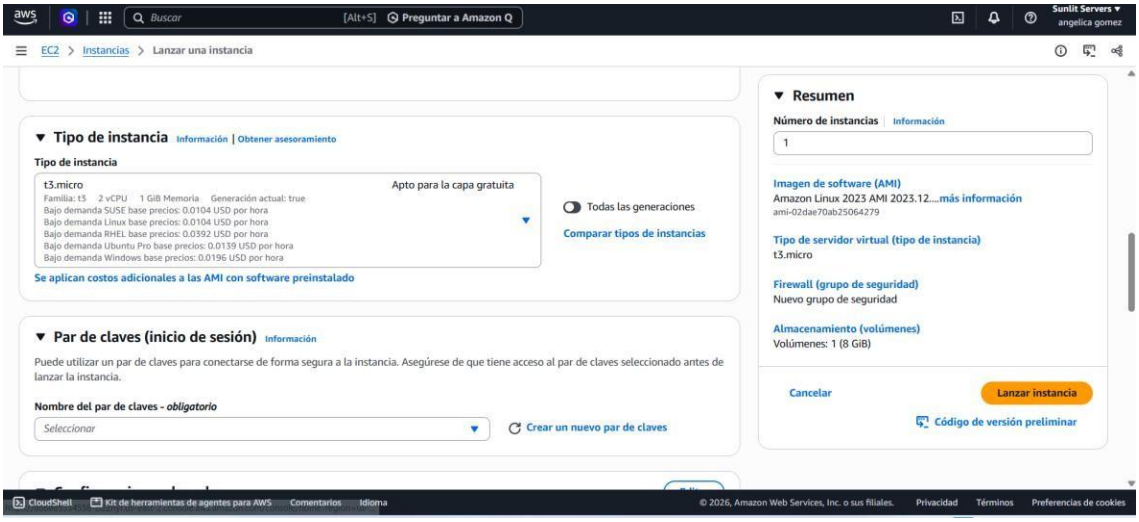
6. Seleccionar el sistema operativo Linux

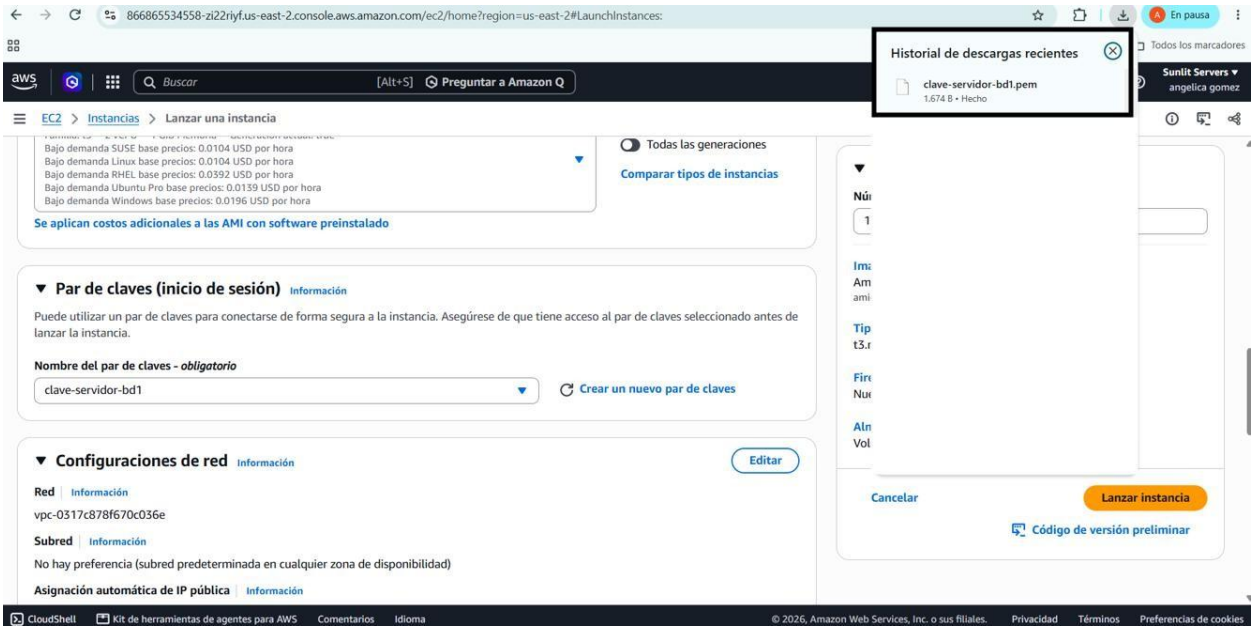


En Application and OS Images (Amazon Machine Image):

- Selecciona Amazon Linux.
- Escoge Amazon Linux 2023 AMI.
- Selecciona la arquitectura x86_64, si está disponible para el tipo de instancia elegido.

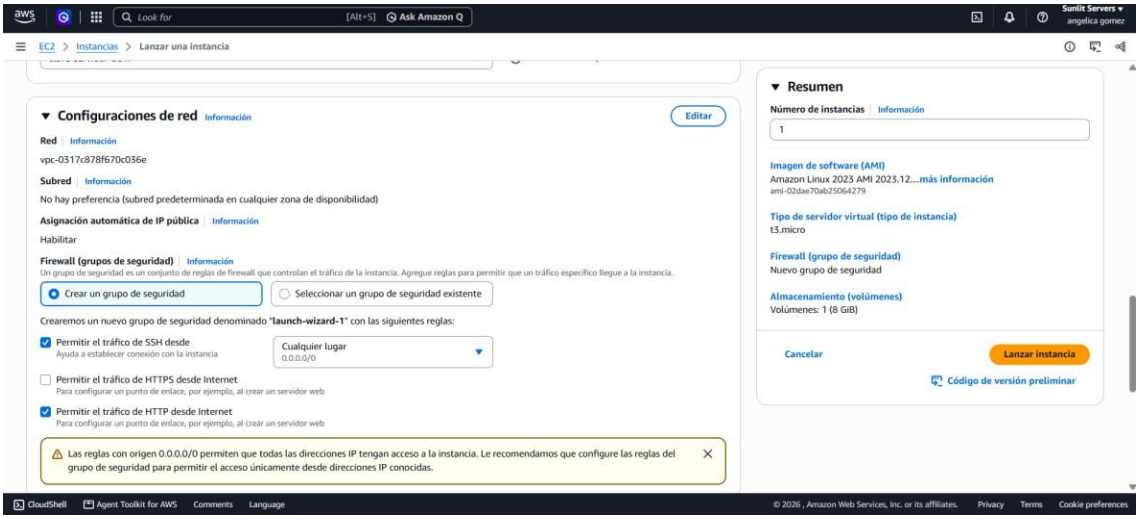
7. Crear el par de claves





- Busca Key pair (login).
- Haz clic en Create new key pair.
- Nombre: clave-servidor-bd1.
- Tipo: RSA.
- Formato: .pem.
- Haz clic en Create key pair.
- Se descargará un archivo similar a: clave-servidor-bd1.pem

8. Configurar la seguridad de red

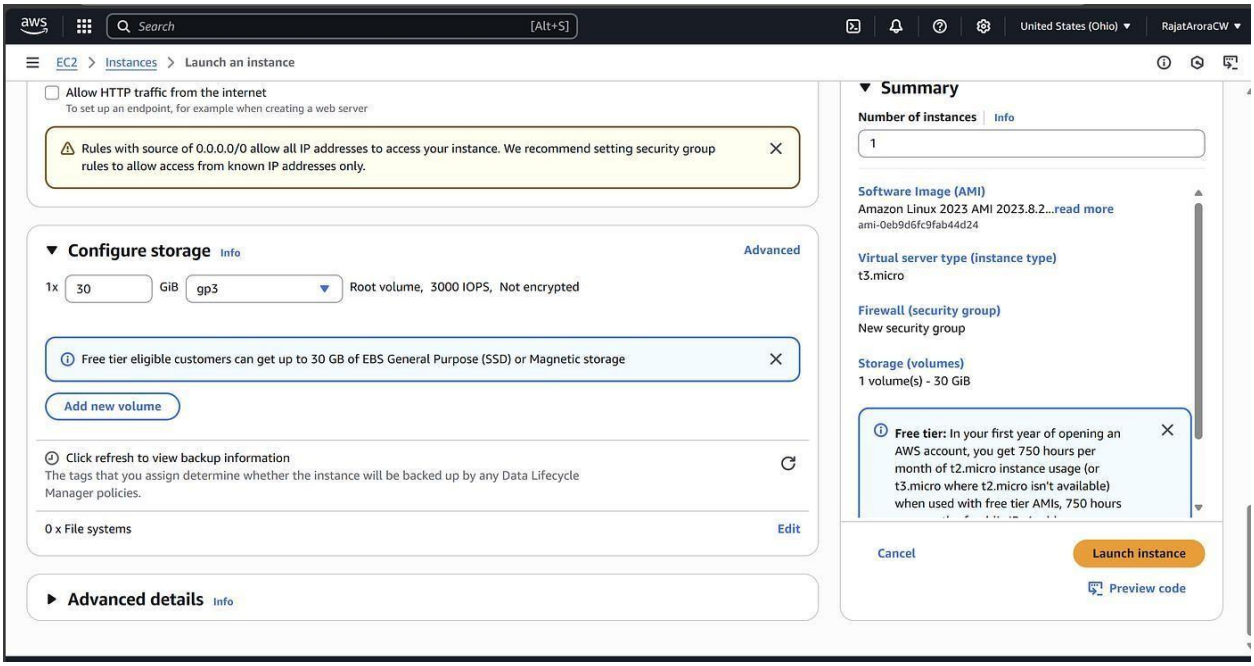


En Network settings:

- Deja seleccionada la VPC predeterminada, si no tienes otra configuración indicada.

- Habilita la asignación de IP pública.
- Crea un nuevo grupo de seguridad.
- En las reglas de entrada selecciona SSH.
- En Source selecciona My IP.

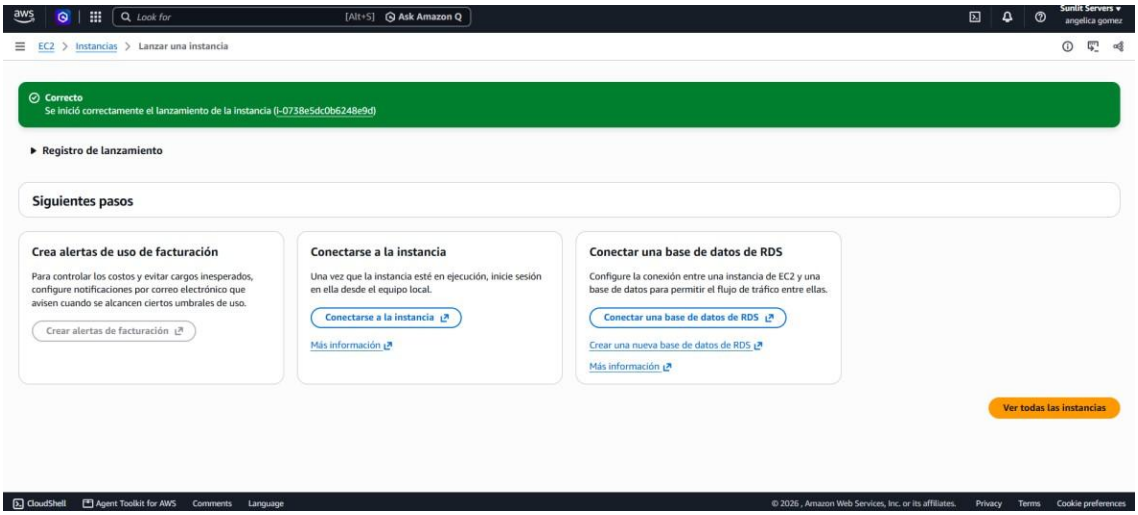
9. Configurar almacenamiento

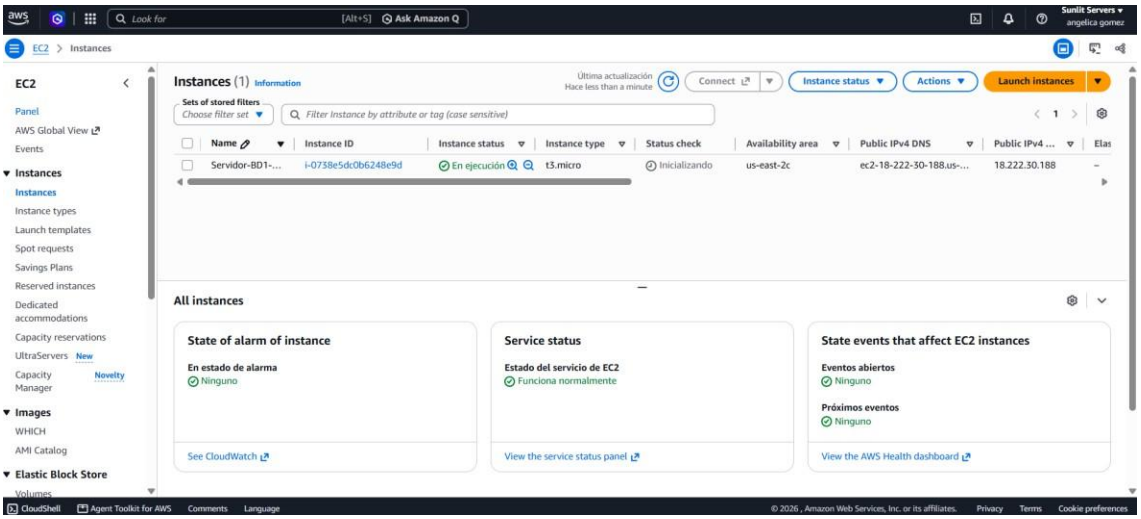


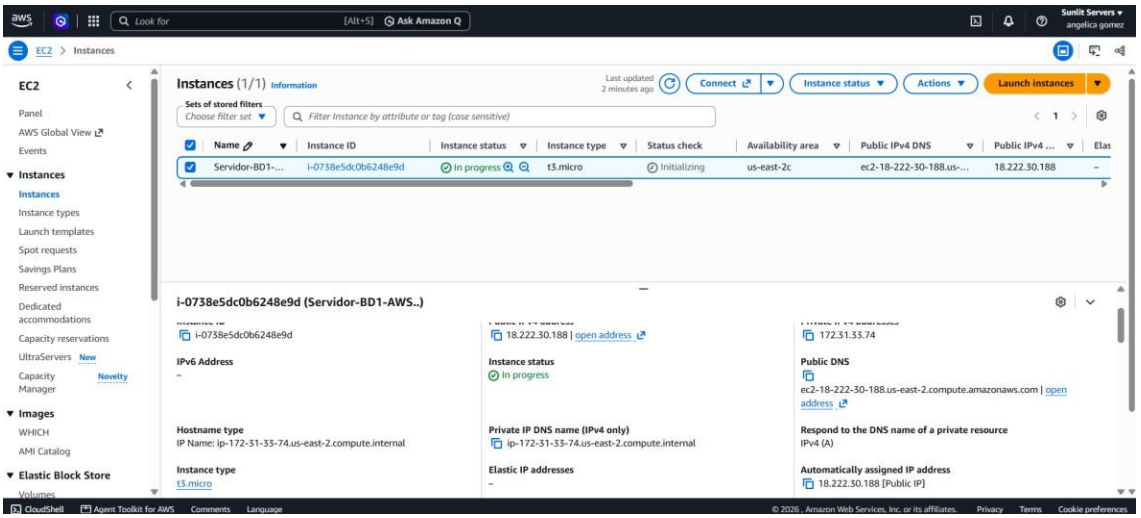
En Configure storage:

- Volumen: 8 GiB como punto de partida.
- Tipo: gp3.
- Mantén los demás valores predeterminados

9. Lanzar la instancia







- Revisa el resumen de configuración.
- Comprueba que el sistema operativo sea Amazon Linux 2023.
- Verifica el tipo de instancia y el grupo de seguridad.
- Haz clic en Launch instance.

10. Conclusión

La instancia EC2 con Ubuntu Server 24.04 LTS cumple lo que pide el enunciado: es una máquina virtual con una distribución Linux, y se administra por SSH. Con ello queda lista la plataforma donde se instalará MySQL, lo que se documenta en el documento de implementación de la RDBMS.

Anexo C: Análisis de selección del RDBMS

1. Para qué sirve este documento

Aquí explicamos qué sistema gestor de bases de datos relacional (RDBMS) vamos a instalar en la máquina virtual y por qué lo elegimos. El enunciado de la Fase 2 pide un análisis de selección y excluye únicamente SQL Server; sugiere MySQL, PostgreSQL y MariaDB, entre otros. Queríamos que la decisión saliera de lo que nuestro modelo realmente necesita, y no de cuál motor suena más popular.

2. El problema que vamos a resolver

El sistema modela el inventario de una tienda de cartas coleccionables de Magic: The Gathering ejemplar por ejemplar: qué cartas tiene, en qué condición está cada copia, cuánto vale según su precio de referencia y si está disponible o apartada. Los clientes registrados pueden reservar ejemplares desde su perfil en la página, con un plazo máximo de 30 días para reclamarlos; vencido el plazo, la reserva se libera automáticamente. Cada entrega de ejemplares a un cliente queda registrada como salida del inventario.

El sistema no maneja ventas, compras, pagos ni facturación. Tiene tres perfiles de usuario: cliente, empleado y administrador. El enunciado completo y los supuestos del modelo están en el archivo Propuesta_Proyecto_BD1.pdf de nuestro repositorio de GitHub.

3. Las opciones que miramos

Evaluamos motores que cumplen la restricción del enunciado y que funcionan sin problemas sobre Linux.

- **MySQL Community Server.** Es un motor robusto, sumamente popular en el desarrollo web y con una amplia documentación orientada a entornos de producción y educativos. Soporta InnoDB, claves compuestas, restricciones CHECK, disparadores (*triggers*) y vistas, integrándose de forma nativa con los lenguajes de la aplicación web.
- **PostgreSQL.** Es excelente en integridad y en cumplimiento estricto del estándar SQL. Es una opción sumamente sólida y técnica, pero para nuestro ecosistema de desarrollo web y despliegue rápido, MySQL nos ofrece una curva más ágil y familiar en el equipo.
- **MariaDB.** Derivado directo de MySQL, con altas prestaciones, aunque optamos por MySQL Community por su estandarización directa y soporte oficial directo en la mayoría de herramientas de despliegue.

4. Nuestra decisión: MySQL Community Server

Elegimos **MySQL Community Server**, instalado sobre nuestra instancia en AWS. Las razones, en orden de importancia para nosotros:

1. **Alineación con el ecosistema web.** MySQL es el estándar de facto para arquitecturas de aplicaciones web ligeras, lo que facilita enormemente la conexión con los controladores y ORMs del mercado sin requerir configuraciones complejas de cliente.

2. **Soporte nativo para los requerimientos del modelo.** Las claves compuestas, las restricciones de integridad referencial (motor InnoDB), las vistas y los disparadores (*triggers*) están plenamente soportados.

3. **Facilidad de despliegue y portabilidad.** Los scripts DDL y DML son limpios, rápidos de ejecutar mediante terminal o herramientas gráficas, y su consumo de recursos en la instancia EC2 es eficiente y predecible.

4. **Suficiencia frente a nuestros volúmenes.** Una tienda, un catálogo de cartas y miles de ejemplares operan con soltura bajo la arquitectura optimizada de InnoDB.

Queremos ser claros con lo que implica esta decisión: aunque MySQL ha mejorado enormemente su soporte para restricciones estrictas y tipos de datos modernos, exige rigor al programar los disparadores para asegurar reglas complejas que cruzan múltiples tablas. Lo asumimos porque su integración con la aplicación web y la facilidad de mantenimiento pesan más para este proyecto.

5. Instalación a grandes rasgos

El procedimiento general para la puesta en marcha del motor en la máquina virtual comprende:

- Conexión por SSH a la instancia EC2 con el sistema operativo base.
- Repositorios e instalación limpia del paquete de MySQL Server.
- Inicio del servicio, configuración de seguridad inicial (mysql_secure_installation) y establecimiento de contraseñas seguras.
- Creación de la base de datos del proyecto, esquemas y asignación de usuarios específicos para la aplicación web.
- Ejecución ordenada de los scripts DDL y DML para poblar el sistema y validar las consultas del curso.

6. Riesgos que vemos

- Que los disparadores complejos presenten bloqueos de concurrencia al intentar reservar múltiples ejemplares de forma simultánea. Se mitiga realizando pruebas de estrés y scripts de validación concurrente.
- Que los permisos de los usuarios queden demasiado abiertos en la aplicación web. Se mitiga aplicando estrictamente el principio de mínimo privilegio en las cuentas creadas en MySQL.

- Que existan diferencias de sintaxis entre las herramientas de diseño gráfico y el motor final. Se previene revisando y depurando manualmente cada sentencia DDL antes de su ejecución definitiva.

12. Conclusión

MySQL Community Server sobre nuestra infraestructura en AWS es el RDBMS idóneo para el proyecto debido a su estabilidad, su perfecta integración con entornos web, su soporte completo para integridad referencial y su agilidad de despliegue. Cumple con todas las exigencias del modelo de cartas coleccionables y nos permite documentar de forma clara la implementación técnica solicitada.

Anexo D: Implementación del RDBMS

1. Introducción

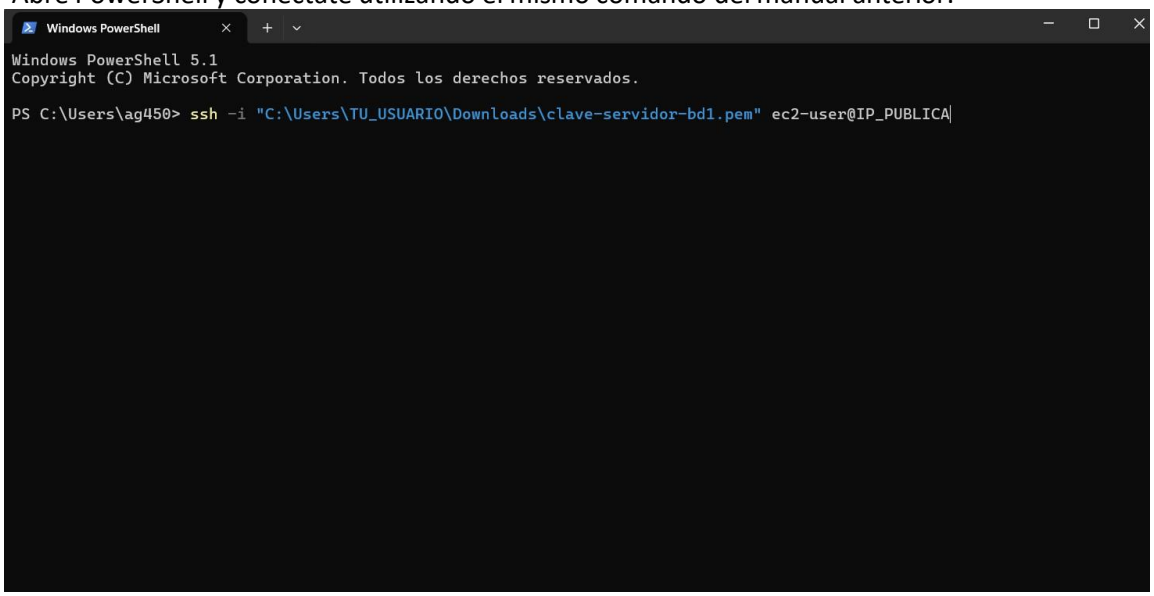
Este documento da un paso a paso de la instalación y configuración de MySQL en la instancia EC2 Magic_Cards, la carga de los scripts de la base de datos y la conexión segura desde el backend que corre en Windows.
Versiones usadas en esta guía:

- MySQL 8.0 paquete mysql-server
- Ubuntu Server 24.04 LTS en una instancia EC2 t3.micro
- MySQL Workbench 8.0 en Windows
- Cliente SSH de Windows (OpenSSH, desde PowerShell)

Prerrequisito: tener la instancia creada y poder conectarse por SSH, como se explica en el documento de implementación de la VM.

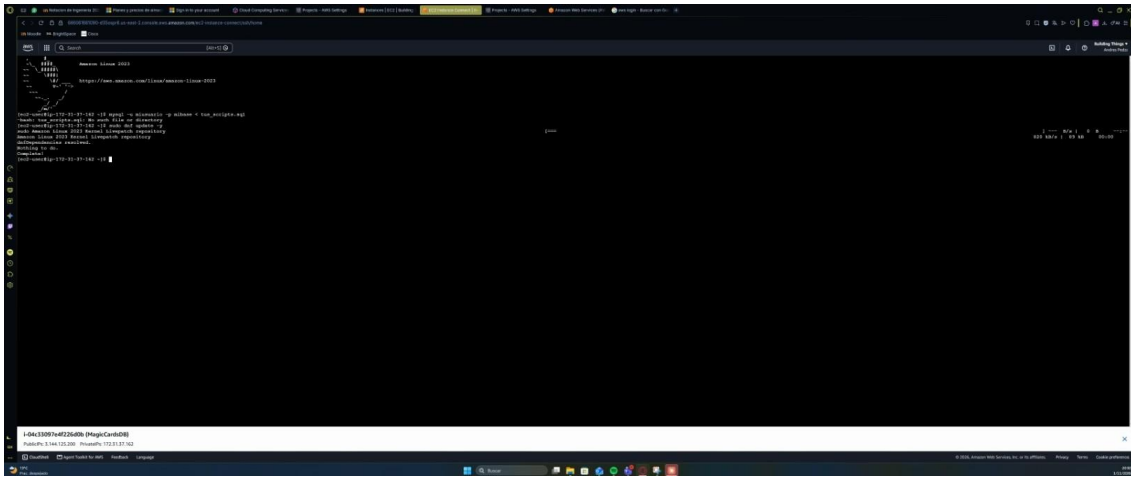
2. Ingresar nuevamente al servidor Linux

Abre PowerShell y conéctate utilizando el mismo comando del manual anterior:

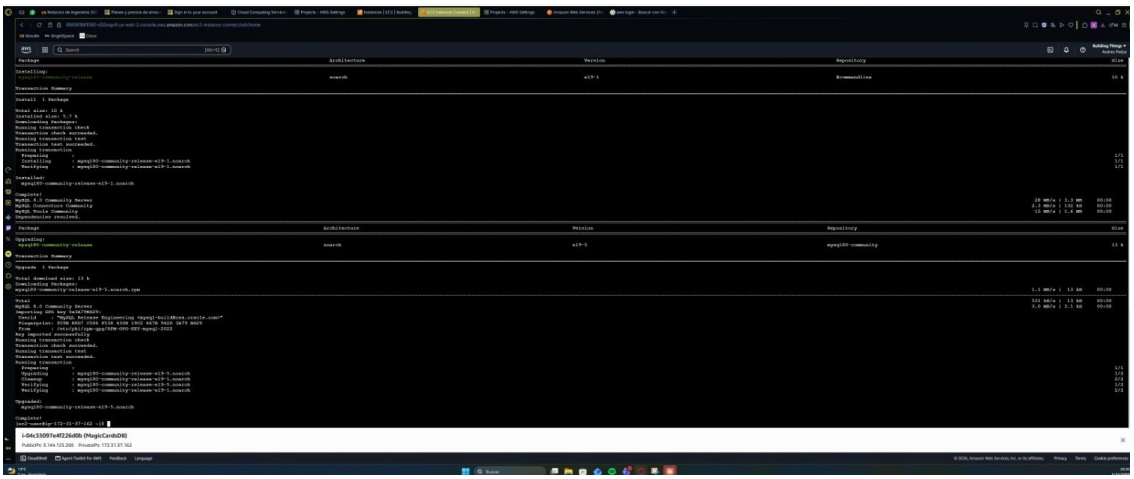


3. Descargar el repositorio oficial de MySQL

Para instalar MySQL Community Server utilizaremos el repositorio oficial de MySQL.
Ejecuta:
sudo dnf install -y https://dev.mysql.com/get/mysql84-community-release-el9-4.noarch.rpm

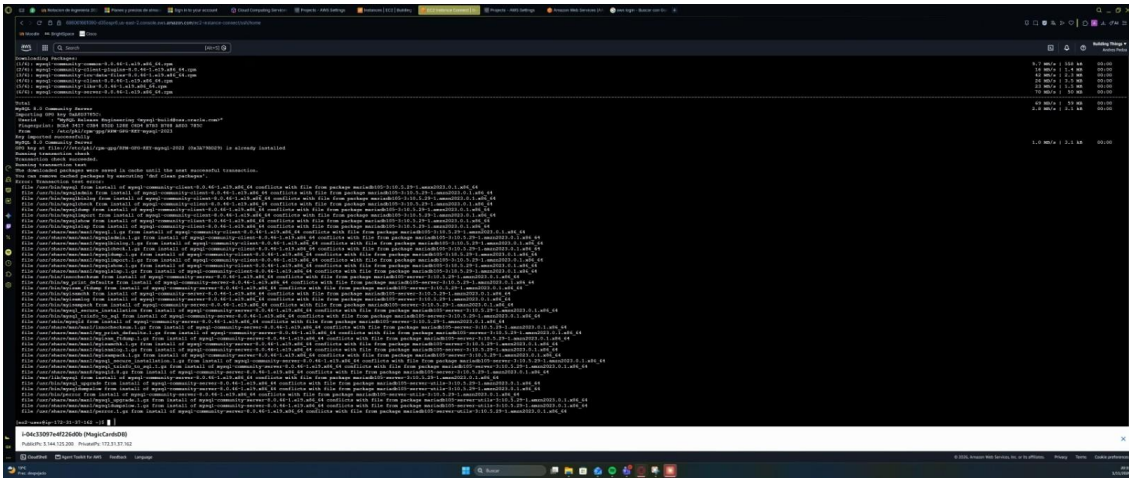


sudo dnf repolist enabled | grep mysql



4. Instalar MySQL Server

Ahora ejecuta:
sudo dnf install mysql-community-server -y



5. Instalar MySQL Server

Ejecuta:
sudo systemctl start mysqld
ESTE COMANDO INICIARA MYSQL

O si prefieres configura el inicio automático:
sudo systemctl enable mysqld
Por último esto permite que MySQL se inicie automáticamente cuando se reinicie la máquina virtual.

13. Conclusión

MySQL 8.0 quedó instalado en la instancia EC2, con la base de datos `magic_cards` cargada y un usuario de aplicación con permisos limitados. El acceso desde Windows se hace por un túnel SSH, de modo que el puerto de la base de datos no queda expuesto a internet. Con esto se cumple el requisito de implementar el RDBMS en una máquina virtual Linux y conectarlo con la aplicación web.

Anexo E: Acta de constitución del proyecto (incluye interesados, cronograma y presupuesto)

1. **Datos generales**

- Proyecto: Sistema Interno de Inventario y Reservas de Cartas Coleccionables.
- Contexto: Proyecto final del curso Bases de Datos 1 (16 % de la nota del semestre), desarrollado durante 14 semanas en el periodo académico 2026-2.
- Docente / Evaluador: Christian Felipe Duarte.
- Repositorio oficial: https://github.com/Ande2323/ProyectoBasesDatosI
- Fecha de inicio: 4 de agosto de 2026.
- Fecha de cierre prevista: 2 de noviembre de 2026.

2. **El problema y por qué lo elegimos**

El sistema modela el inventario de una tienda de cartas coleccionables de *Magic: The Gathering* ejemplar por ejemplar: qué cartas tiene, en qué condición está cada copia, cuánto vale según su precio de referencia y si está disponible o apartada. Los clientes registrados pueden reservar ejemplares desde su perfil en la página web, con un plazo máximo de 30 días para reclamarlos; vencido el plazo, la reserva se libera automáticamente. Cada entrega de ejemplares a un cliente queda registrada como salida del inventario.

El sistema no maneja ventas, compras, pagos ni facturación. Cuenta con tres perfiles de usuario: cliente, empleado y administrador.

Elegimos este problema porque la información de una tienda de coleccionables suele estar dispersa, generando riesgos de negocio concretos (reservar dos veces el mismo ejemplar, perder trazabilidad de entregas o desconocer el valor del inventario). Además, posee la complejidad técnica ideal para aplicar los conceptos del curso (17 entidades, relaciones 1:1, 1:N y N:M, entidades débiles, una relación recursiva, atributos derivados y normalización a 3FN), manteniendo un alcance controlado.

3. **Objetivos**

- Objetivo general: Diseñar, implementar y documentar una base de datos relacional normalizada hasta la Tercera Forma Normal (3FN), desplegada en una máquina virtual Linux en AWS y consultable desde una aplicación web de demostración.
- Objetivos específicos:

- Modelar el dominio mediante diagramas Entidad-Relación (notaciones Peter Chen y Crow's Foot) abarcando 17 entidades en Oracle SQL Developer Data Modeler.
- Construir el modelo relacional en 3FN y formalizarlo en el diccionario de datos.
- Formular 50 sentencias de álgebra relacional que respondan a las consultas clave del sistema.
- Seleccionar, justificar e implementar el sistema operativo y el RDBMS en una máquina virtual administrada (AWS EC2).
- Trasladar el modelo lógico al físico mediante scripts DDL, DML y objetos avanzados (vistas, triggers), documentados con su respectivo README.
- Desarrollar una aplicación web sencilla conectada a la base de datos para la lectura de datos en tiempo real.
- Mantener el control de versiones en GitHub con aportes equitativos y visibles de los tres integrantes.
- Elaborar el documento final, la presentación ejecutiva y un video de demostración de máximo 15 minutos.

4. Alcance

4.1 Lo que sí incluimos

- Catálogo de cartas (colores, tipos, ediciones, rarezas, artistas, idiomas y linaje de reimpresiones).
- Inventario por ejemplar (condición física, ubicación en el local, fecha de ingreso y estado).
- Historial de precios de referencia y cálculo de precio sugerido.
- Tres perfiles de usuario (cliente, empleado y administrador).
- Módulo de reservas con vigencia de 30 días y liberación automática.
- Registro de salidas identificando cliente y empleado responsable.
- Consultas y agregaciones: valor del inventario, disponibilidad real y vencimiento de reservas.
- Aplicación web de demostración para lectura de datos.

1.2 Lo que dejamos por fuera

- Ventas, compras, consignaciones, intercambios, pagos y facturación.
- Multisucursales, monedas extranjeras, ventas en línea y cotizaciones en tiempo real.
- Formatos de juego, mazos, reglas de interacción y erratas de cartas.
- Gestión de proveedores, devoluciones, garantías y envíos a domicilio.
- Notificaciones automáticas, prorrogas o penalizaciones económicas.

5. Responsabilidades funcionales del sistema

- Clientes: Consultan el catálogo, se registran y gestionan sus propias reservas.
- Empleados: Registran ingreso de ejemplares, actualizan condición/ubicación, cargan precios de referencia, procesan salidas y marcan reservas como reclamadas o vencidas.
- Administrador: Posee permisos globales (gestión de catálogos, usuarios, roles y personal).
- Motor / Sistema: Calcula dinámicamente precios sugeridos, determina el valor del inventario y ejecuta la liberación automática de ejemplares al vencer sus reservas.

6. Entregables del proyecto

- Fase 1: Propuesta, documento de supuestos, modelos ER (Chen y Crow's Foot), diccionario de datos y documento de álgebra relacional (50 sentencias).
- Fase 2: Documentos de selección e implementación del SO y RDBMS, scripts DDL/DML/varios con README, aplicación web conectada, repositorio Git y video de demostración.
- Transversales: Acta de constitución, matriz de interesados, EDT/WBS, cronograma, presupuesto y acta de cierre.

7. Decisiones técnicas y su justificación

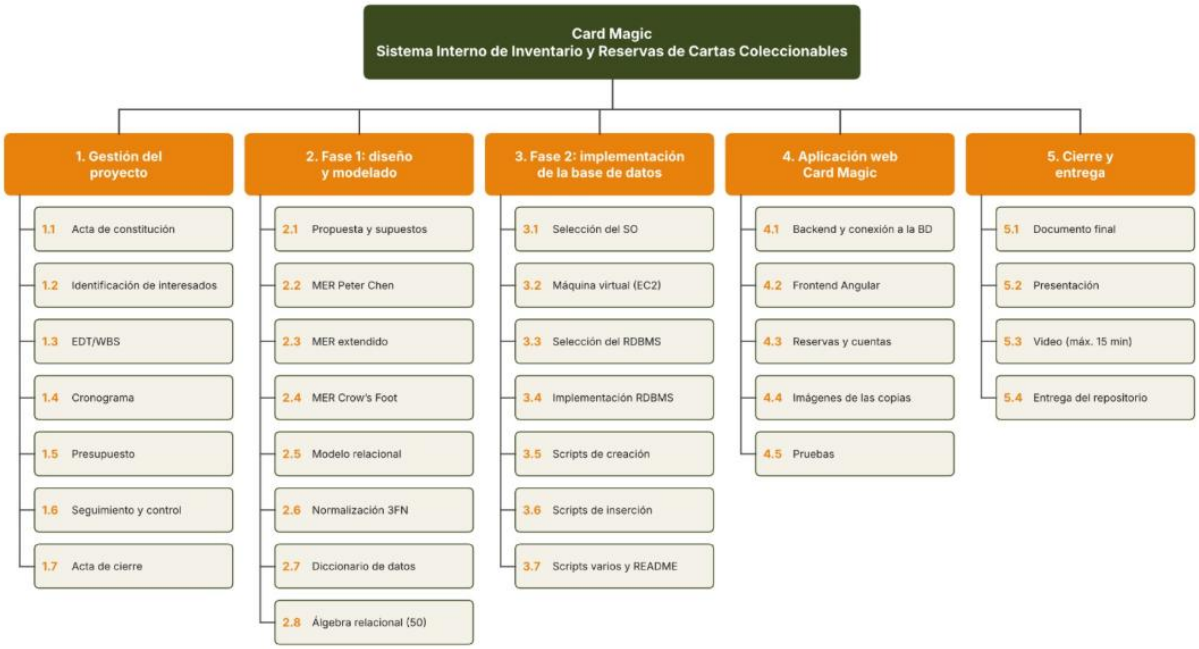
- Sistema operativo: *Oracle Linux 9*. Plataforma empresarial estable, segura y optimizada para entornos de servidores de base de datos.
- Infraestructura: *Instancia EC2 en AWS (IaaS)*. Permite administrar directamente el sistema operativo y el motor de base de datos, cumpliendo la exigencia de documentar una VM autogestionada sin recurrir a servicios administrados tipo RDS.
- RDBMS: *MySQL Community Server (Engine InnoDB)*. Motor relacional de alto rendimiento, ideal para integración con aplicaciones web, con soporte completo para

claves compuestas, integridad referencial, disparadores (triggers), vistas y tareas programadas.

- Diseño y Normalización: Aplicación estricta de la 3FN desde el modelado para eliminar dependencias transitivas (ej. tabla independiente para CONDICION).
- Control de versiones: GitHub para garantizar la trazabilidad de los aportes de todo el equipo.

8. Estructura de Desglose del Trabajo (EDT / WBS)

La EDT/WBS organiza las actividades del proyecto en componentes principales:



9. Hitos y Cronograma General (14 Semanas)

- Semanas 1 y 2: Inicio, propuesta de proyecto, supuestos preliminares y acta de constitución.
- Semanas 3 a 6: Elaboración de modelos ER, modelo relacional en Data Modeler y diccionario de datos.
- Semanas 6 a 8: Formulación y validación de las 50 sentencias de álgebra relacional.
- Semanas 8 a 10: Despliegue de la instancia AWS EC2, instalación de MySQL y redacción de documentos de arquitectura.
- Semanas 10 a 12: Construcción y prueba de scripts DDL, DML, triggers, vistas y documentación README.

- Semanas 12 a 14: Desarrollo de aplicación web, pruebas end-to-end, grabación del video, presentación y acta de cierre.

10. Equipo de trabajo y roles

- Angélica María Gómez Calderón (Código: 1000858150) — *Diseño de BD, Scripts SQL y Documentación Técnica.*
- Andrés Camilo Pedraza Reyes (Código: 1098682948) — *Infraestructura AWS/Linux, RDBMS y Repositorio Git.*
- Daniel Felipe Gómez Veru (Código: 1021632658) — *Desarrollo de Aplicación Web e Integración de Consultas.*

Todos los integrantes participan activamente en las entregas teóricas, la revisión de modelos y la presentación del video final.

11. Interesados clave (Stakeholders)

- Christian Felipe Duarte (Docente / Evaluador): Interesado principal; define requerimientos académicos y evalúa el cumplimiento técnico.
- Equipo de Desarrollo (Estudiantes): Ejecutores del proyecto con el objetivo de aprobar la asignatura y construir una solución funcional.
- Usuarios Finales Ficticios (Clientes, Empleados, Administrador): Guían los requerimientos de usabilidad y seguridad del sistema.

12. Presupuesto preliminar

- Licencias de Software: $0 USD. Se utiliza software libre y de código abierto (Oracle Linux 9, MySQL Community, Data Modeler, Git/GitHub).
- Infraestructura Cloud (AWS EC2): Estimado en $0 USD mediante la utilización de créditos o capa gratuita de AWS. Se implementa una rutina de apagado de la instancia fuera de horarios de trabajo para evitar sobrecostos operativos.

13. Riesgos iniciales y mitigación

- Complejidad del modelo vs. Tiempo disponible: Mitigado acotando el alcance funcional mediante supuestos claros y priorizando lecturas en la web.
- Sobrecostos por instancias encendidas en AWS: Mitigado mediante monitoreo semanal del panel de facturación y apagado programado de la VM.

- Fallas en la regla de negocio de reservas concurrentes: Mitigado mediante pruebas de estrés sobre los disparadores (triggers) creados en MySQL.
- Desalineación en entregas del equipo: Mitigado con reuniones semanales y control estricto de commits en GitHub.

14. Criterios de éxito del proyecto

- Modelo relacional que cumple con más de 8 entidades (17 implementadas) y normalización 3FN comprobada.
- Ejecución correcta y validada de 50 sentencias de álgebra relacional expresadas en SQL.
- Servidor en AWS totalmente funcional, ejecutable de manera reproducible mediante la documentación entregada.
- Aplicación web de demostración conectada exitosamente a la base de datos MySQL.
- Historial de commits equilibrado en GitHub y participación equitativa en el video de demostración.

Anexo F: Acta de cierre

1.Identificación del proyecto

- Nombre del proyecto: Sistema Interno de Inventario y Reservas de Cartas Coleccionables (*Magic_Cards*).
- Asignatura: Bases de Datos 1 (Periodo 2026-2).
- Fecha de inicio: 4 de agosto de 2026.
- Fecha de finalización: 2 de noviembre de 2026.
- Integrantes del equipo: Angélica María Gómez Calderón, Andrés Camilo Pedraza Reyes y Daniel Felipe Gómez Veru.
- Docente / Evaluador: Christian Felipe Duarte.
- Repositorio del proyecto: https://github.com/Ande2323/ProyectoBasesDatosI

2. Resumen ejecutivo

El proyecto consistió en el diseño, implementación y documentación de un sistema de información para modelar el inventario de una tienda especializada en cartas coleccionables de *Magic: The Gathering*.

Se desarrolló el Modelo Entidad-Relación Extendido (MERE) como base conceptual, su transformación al modelo relacional normalizado hasta la Tercera Forma Normal (3FN), y su posterior implementación física en MySQL Server sobre una máquina virtual con Oracle Linux 9 en AWS EC2, conectada a una aplicación web de demostración para consulta de datos en tiempo real.

El sistema permite gestionar la existencia de ejemplares, su condición física, valoración según precios de referencia y su estado de disponibilidad/reserva. Administra reservas de clientes con vencimiento a 30 días y liberación automática, así como la trazabilidad de salidas de inventario con responsable registrado.

3. Objetivos alcanzados

- Diseño conceptual y lógico: Se construyó el modelo MERE representando la operación real de la tienda según los supuestos definidos, derivando en un modelo relacional en 3FN y su diccionario de datos.
- Implementación funcional: Se desplegó el servidor de base de datos en AWS EC2 y se conectó con la aplicación web de demostración.
- Gestión integral de catálogos e inventario: Control total sobre cartas, ediciones, impresiones, ejemplares, historial de precios, reservas y salidas.

- Derivación de métricas: Capacidad de consulta para el valor total del inventario, disponibilidad real, vencimiento de reservas y rotación.

4. Alcance y cumplimiento

- Alcance original: Cobertura de diseño de modelos, diccionario de datos, 50 sentencias de álgebra relacional, scripts DDL/DML, arquitectura de servidor en la nube y aplicación web.
- Alcance ejecutado: Se completó el 100 % de los 14 entregables comprometidos en las dos fases y los transversales, sin ningún pendiente funcional al momento del cierre.

5. Estado final de los entregables

Entregables de la Fase 1

Entregable	**Estado**	**Encontrarlo en…**
Documento de propuesta del proyecto y supuestos	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**
Modelo Entidad-Relación con notación Peter Chen	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**
Modelo Entidad-Relación Extendido (MERE)	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**
Modelo Entidad-Relación con notación Barker o Crow's Foot	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**
Modelo Relacional con notación Barker o Crow's Foot	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**
Diccionario de datos	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**
Documento de implementación de álgebra relacional (mínimo 50 ejercicios)	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 01**

Entregables de la Fase 2

Entregable	**Estado**	**Encontrarlos en…**
Documento de análisis de selección de sistema operativo	COMPLETADO	C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02\Documentacion
Documento de implementación de la máquina virtual (instancia EC2 en AWS)	COMPLETADO	C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02\Documentacion
Documento de análisis de selección del RDBMS	COMPLETADO	C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02\Documentacion
Documento de implementación del RDBMS	COMPLETADO	C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02\Documentacion
Scripts con README (creación de objetos, inserción de registros y scripts varios)	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02**
Conexión de la aplicación web a la base de datos relacional	COMPLETADO	**GITHUB**
Repositorio Git con la aplicación y los scripts	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02 + GIT HUB**
Video demostrativo (máximo 15 minutos, todos los integrantes, audio y cámara)	COMPLETADO	**C:\Users\ag450\Desktop\Proyecto_Final_BD1\FASE 02**

Entregables Transversales

Entregable	Estado	Encontrarlo en…
Acta De Cierre	**COMPLETADO**	
Acta de constitución del proyecto (Identificación de interesados,	**COMPLETADO**	

6. Resultados y logros técnicos

- Modelo conceptual y relacional: Esquema MERE de 17 entidades (15 regulares y 2 débiles) y 20 relaciones, incorporando una relación recursiva con atributo propio, atributos derivados y compuestos, normalizado a 3FN.
- Esquema físico robusto: Implementación de 20 tablas en MySQL con claves primarias compuestas, claves foráneas e integridad referencial garantizada para relaciones 1:1 y 1:N.
- Scripts limpios y ordenados: Automatización completa de la creación de objetos, datos de prueba y roles/usuarios de la aplicación mediante scripts DDL y DML ejecutables en servidor limpio.
- Despliegue y seguridad Cloud: Base de datos instalada en una instancia AWS EC2 administrada (AWS AMAZON / MySQL 8.0), garantizando conexiones seguras y manteniendo los puertos cerrados a internet.
- Documentación técnica completa: Entrega detallada de guías de arquitectura, selección y despliegue de la infraestructura.

7. Compromisos post-cierre

- El equipo mantendrá el repositorio en GitHub de manera pública y permanente con todo el código, scripts y documentación.
- Se conservará un respaldo completo en frío (mysqldump) y de los scripts fuera de la nube para respaldar el trabajo final.
- Tras concluir la sustentación académica, se detendrá/eliminará la instancia AWS EC2 para evitar consumo innecesario de créditos o cobros por uso.
- Disponibilidad del equipo para atender consultas académicas o revisiones adicionales del docente.

8. Lecciones aprendidas

Aspectos positivos

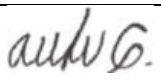
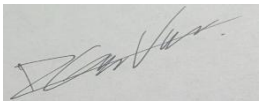
- Metodología por fases: La clara separación entre el modelado conceptual y la implementación física facilitó una transición estructurada.
- Documento de supuestos como guía: Funcionó como la "única fuente de verdad" para resolver dudas sobre reglas de negocio y restricciones del esquema.
- Uso estricto de Git: El control de versiones en GitHub fue clave para integrar los aportes técnicos del equipo.
- Pruebas en entorno limpio: Probar los scripts desde cero previno fallos sintácticos y de orden de dependencias en las claves foráneas.

Oportunidades de mejora

- Planificación temporal: Asignar mayor margen de tiempo inicial a la fase de instalación y configuración de la infraestructura física.
- Documentación continua: Redactar los entregables de forma progresiva a la par del código, evitando la acumulación de documentación al final del ciclo.
- Evaluación temprana de la nube: Monitorear desde el primer día los costos, vigencia de créditos y restricciones de licencias de las imágenes en los Marketplaces de AWS.

9. Cierre y aceptación formal

Con la firma de este documento y la entrega de la totalidad de los componentes en el repositorio oficial, se da por finalizado satisfactoriamente el desarrollo del proyecto Magic_Cards.

Nombre	Cargo / Rol	Firma	Fecha
Christian Felipe Duarte	Asesor / Docente		
Angélica María Gómez Calderón	Integrante del equipo		2/11/2026
Andrés Camilo Pedraza Reyes	Integrante del equipo		2/11/2026
Daniel Felipe Gómez Veru	Integrante del equipo		2/11/2026

9. REFERENCIAS

1. Marín, R. (2024, abril 8). *Los gestores de bases de datos más usados en la actualidad*. Canal Informática y TICS. https://www.inesem.es/revistadigital/informatica-y-tics/los-gestores-de-bases-de-datos-mas-usados
2. (S/f). Amazon.com. Recuperado el 8 de noviembre de 2026, de https://docs.aws.amazon.com/es_es/cli/latest/userguide/getting-started-install.html#getting-started-install-instructions
3. Fernandoing [@fernandoing]. (s/f). *Aprender AWS desde cero - 15 - EC2 - Interfaz de línea de comandos de AWS* [[Object Object]]. Youtube. Recuperado el 8 de noviembre de 2026, de https://www.youtube.com/watch?v=Ic3fNppQYyc
4. *Introduction to relational algebra in DBMS*. (2015, julio 2). GeeksforGeeks. https://www.geeksforgeeks.org/dbms/introduction-of-relational-algebra-in-dbms/
5. López, M. M. (2022, septiembre 26). ¿Cómo crear un glosario en Data Modeler? [Guía rápida 2026]. *KeepCoding Bootcamps*. https://keepcoding.io/blog/glosario-en-data-modeler/
6. Gascón, O. [@todopmp]. (s/f). *Estructura de desglose de Trabajo / Work Breakdown Structure EDT/WBS* [[Object Object]]. Youtube. Recuperado el 8 de noviembre de 2026, de https://www.youtube.com/watch?v=MhFv25AMuOk
7. *Schema, DDL and DML*. (s/f). Hasura.Io. Recuperado el 8 de noviembre de 2026, de https://hasura.io/learn/database/mysql/core-concepts/3-schema-ddl-dml/
8. Youtube. (s/f). - *YouTube* [[Object Object]]. Recuperado el 8 de noviembre de 2026, de https://www.youtube.com/watch?v=SZ5mtoLNrQA